

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

03

자바의 기초 - 자료형



03-1 숫자

03-2 불

03-3 문자

03-4 문자열

03-1 숫자

정수

- 자바에서 정수를 표현하는 자료형 – **int**, **long**

자료형	표현 범위
int	-2147483648 ~ 2147483647
long	-9223372036854775808 ~ 9223372036854775807

- 사용 예

```
int age = 10;
long countOfStar = 8764827384923849L;
```

이렇게 붙여 준다.

  - long 자료형 변수에 값을 대입할 때 int 자료형의 최댓값인 2147483647보다 크면 8764827384923849L처럼 끝에 접미사로 L 자를 붙여주어야 함

03-1 숫자

실수

- 자바에서 실수를 표현하는 자료형 – **float**, **double**

자료형	표현 범위
float	$-3.4 * 10^{38} \sim 3.4 * 10^{38}$
double	$-1.7 * 10^{308} \sim 1.7 * 10^{308}$

- 사용 예

```
float pi = 3.14F;
double morePi = 3.14159265358979323846;
```

  - 기본값은 double이므로 float 변수에 값을 대입할 때 끝에 접미사로 F나 f를 붙여 주지 않으면 컴파일 오류 발생

03-1 숫자

8진수와 16진수

- 자바에서 8진수와 16진수를 표현하는 자료형 – **int**
- 숫자 0으로 시작하면 8진수, 숫자 0과 알파벳 x로 시작하면 16진수

- 사용 예

```
int octal = 023;    // 십진수: 19
int hex = 0xC3;     // 십진수: 195
```

  - 023과 0xC3에 해당하는 십진수 값은 각각 19, 195

03-1 숫자

숫자 연산

- +, -, *, / 기호를 이용하여 두 숫자 간의 사칙연산을 수행

Do it! 코딩해 보세요 — Sample.java

```
public class Sample {
    public static void main(String[] args) {
        int a = 10;
        int b = 5;
        System.out.println(a+b);
        System.out.println(a-b);
        System.out.println(a*b);
        System.out.println(a/b);
    }
}
```

실행 결과

```
15
5
50
2
```

03-1 숫자

▪ 숫자 연산

- +, -, *, / 기호를 이용하여 두 숫자 간의 사칙연산을 수행

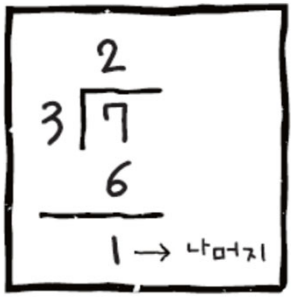
Do it! 코딩해 보세요

```
public class Sample {
    public static void main(String[] args) {
        System.out.println(7 % 3);  // 1 출력
        System.out.println(3 % 7);  // 3 출력

    }
}
```

실행 결과

```
1
3
```



03-1 숫자

▪ 증감 연산

- ++, -- 기호(증감 연산자)를 이용하여 값을 증가 또는 감소시킴

Do it! 코딩해 보세요

```
int i = 0;
int j = 10;
i++;
j--;

System.out.println(i);  // 1 출력
System.out.println(j);  // 9 출력
```

실행 결과

```
1
9
```

Do it! 코딩해 보세요

```
int i = 0;
System.out.println(i++);  // 0 출력
System.out.println(i);  // 1 출력
```

실행 결과

```
0
1
```

- ++ 연산자의 위치에 따라 결괏값이 다름

03-2 불

▪ 불 연산

- 불(boolean) 자료형 - 참(true) 또는 거짓(false)의 값을 갖는 자료형

```
boolean isSuccess = true;
boolean isTest = false;
```

- 불 연산은 참 또는 거짓을 판단하는 연산
- if 문과 같은 조건문에 사용하거나 불 자료형에 대입할 수 있음

```
2 > 1            // 참
1 == 2           // 거짓
3 % 2 == 1       // 참(3을 2로 나눈 나머지는 1이므로 참이다.)
"3".equals("2")  // 거짓
```

03-2 불

▪ 불 연산 사용 예 - 조건문

- 불 연산은 보통 조건문의 판단 기준으로 많이 사용

Do it! 코딩해 보세요

```
int base = 180;
int height = 185;
boolean isTall = height > base;

if (isTall) {
    System.out.println("키가 큽니다.");
}
```

실행 결과

```
키가 큽니다.
```

Do it! 코딩해 보세요

```
int i = 3;
boolean isOdd = i % 2 == 1;
System.out.println(isOdd);  // true 출력
```

실행 결과

```
true
```

03-3 문자

▪ 문자 자료형 - char

```
char a1 = 'a';
```

문잣값 앞뒤를 단일 인용 부호 ' '로 감싸 주어야 한다.

- char 자료형은 활용성이 적지만 문잣값을 표현하는 방식은 다양함으로 주의해야 함

Do it! 코딩해 보세요

```
char a1 = 'a';  // 문자로 표현
char a2 = 97;  // 아스키코드로 표현
char a3 = '\u0061';  // 유니코드로 표현

System.out.println(a1);  // a 출력
System.out.println(a2);  // a 출력
System.out.println(a3);  // a 출력
```

실행 결과

```
a
a
a
```

- 세 종류의 코드는 a라는 문자를 똑같이 출력

03-4 문자열

▪ 문자열 – String

Do it! 코딩해 보세요

```
String a = "Happy Java";
String b = "a";
String c = "123";
```

Do it! 코딩해 보세요

```
String a = new String("Happy Java");
String b = new String("a");
String c = new String("123");
```

- 문자열의 앞과 뒤는 쌍따옴표(" ")로 감쌈
- new 키워드는 객체를 만들 때 사용
- 문자열을 표현할 때는 가급적이면 리터럴(literal) 표기 방식을 사용하는 것이 좋음
  - 리터럴 표기 방식 – 객체를 생성하지 않고 고정된 값을 그대로 대입하는 방법(첫 번째 방식)

03-4 문자열

원시(primitive) 자료형

- int, long, double, float, boolean, char 자료형이 해당
- new 키워드로 값을 생성할 수 없고, 리터럴 표기 방식으로만 값을 세팅할 수 있음

```
boolean result = true;
char a = 'A';
int i = 100000;
```

- String은 리터럴 표기 방식을 사용할 수 있지만 원시 자료형에 포함되지 않음

03-4 문자열

원시(primitive) 자료형

Wrapper 클래스

- 원시 자료형을 대신하여 사용할 수 있는 자료형
- 객체지향 프로그래밍의 모든 기능을 활용할 수 있게 해줌

원시 자료형	Wrapper 클래스
int	Integer
long	Long
double	Double
float	Float
boolean	Boolean
char	Char

- ArrayList, HashMap, HashSet 등은 데이터를 생성할 때 원시 자료형 대신 그에 대응하는 Wrapper 클래스를 사용해야 함

03-4 문자열

문자열 내장 메서드

equals

- 문자열 2개가 같은지를 비교하여 결괏값을 리턴

Do it! 코딩해 보세요

```
String a = "hello";
String b = "java";
String c = "hello";
System.out.println(a.equals(b)); // false 출력
System.out.println(a.equals(c)); // true 출력
```

실행 결과

```
false
true
```

- 문자열의 값을 비교할 때는 반드시 == 연산자 대신 equals를 사용

03-4 문자열

▪ 문자열 내장 메서드

▪ indexOf

- 문자열에서 특정 문자열이 시작되는 위치(인덱스값)를 리턴
  - 자바에서는 숫자를 0부터 세는 것을 유의

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.indexOf("Java"));  // 6 출력
```

실행 결과

```
6
```

03-4 문자열

▪ 문자열 내장 메서드

▪ contains

- 문자열에서 특정 문자열이 포함되어 있는지 여부를 리턴

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.contains("Java"));  // true 출력
```

실행 결과

```
true
```

▪ charAt

- 문자열에서 특정 위치의 문자를 리턴

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.charAt(6));  // "J" 출력
```

실행 결과

```
J
```

03-4 문자열

▪ 문자열 내장 메서드

▪ replaceAll

- 문자열에서 특정 문자열을 다른 문자열로 바꿀 때 사용

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.replaceAll("Java", "World"));  // "Hello World" 출력
```

실행 결과

```
Hello World
```

▪ substring

- 문자열에서 특정 문자열을 뽑아낼 때 사용

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.substring(0, 4));  // "Hell" 출력
```

실행 결과

```
Hell
```

03-4 문자열

▪ 문자열 내장 메서드

▪ toUpperCase

- 문자열을 모두 대문자로 변경할 때 사용

Do it! 코딩해 보세요

```
String a = "Hello Java";
System.out.println(a.toUpperCase());  // "HELLO JAVA" 출력
```

실행 결과

```
HELLO JAVA
```

▪ split

- 문자열을 특정한 구분자로 나누어 문자열 배열로 리턴

Do it! 코딩해 보세요

```
String a = "a:b:c:d";
String[] result = a.split(":"); // result는 {"a", "b", "c", "d"}
```

03-4 문자열

문자열 포매팅(formatting)

- 문자열 안에 어떤 값을 삽입하는 방법

- **숫자 바로 대입하기**
  - String.format 메서드 사용
    - 숫자를 넣는 문자열 포맷 코드 %d

Do it! 코딩해 보세요

```
System.out.println(String.format("I eat %d apples.", 3));  // "I eat 3 apples." 출력
```

실행 결과

```
I eat 3 apples.
```

03-4 문자열

문자열 포매팅(formatting)

문자열 바로 대입하기

- String.format 메서드 사용
  - 문자열을 넣는 문자열 포맷 코드 %s

Do it! 코딩해 보세요

```
System.out.println(String.format("I eat %s apples.", "five"));  // "I eat five apples." 출력
```

실행 결과

```
I eat five apples.
```

03-4 문자열

문자열 포매팅(formatting)

숫자값을 나타내는 변수 대입하기

Do it! 코딩해 보세요

```
int number = 3;
System.out.println(String.format("I eat %d apples.", number));  // "I eat 3 apples." 출력
```

실행 결과

```
I eat 3 apples.
```

값을 2개 이상 넣기

Do it! 코딩해 보세요

```
int number = 10;
String day = "three";
System.out.println(String.format("I ate %d apples. So I was sick for %s days.", number,
day));
```

실행 결과

```
I ate 10 apples. So I was sick for three days.
```

03-4 문자열

▪ 문자열 포맷 코드

▪ 문자열 포맷 코드의 종류

종류	설명
%s	문자열(String)
%c	문자 1개(character)
%d	정수(Integer)
%f	부동 소수(floating-point)
%o	8진수
%x	16진수
%%	특수 문자 %

Do it! 코딩해 보세요

```
System.out.println(String.format("I have %s apples.", 3));
System.out.println(String.format("rate is %s.", 3.234));
```

실행 결과

```
I have 3 apples.
rate is 3.234.
```

03-4 문자열

▪ 문자열 포맷 코드

▪ 문자열 포맷 코드 응용하기

1. 정렬과 공백 표현하기

Do it! 코딩해 보세요

```
System.out.println(String.format("%10s", "hi."));  // "      hi." 출력
```

실행 결과

```
      hi.
```



03-4 문자열

▪ 문자열 포맷 코드

▪ 문자열 포맷 코드 응용하기

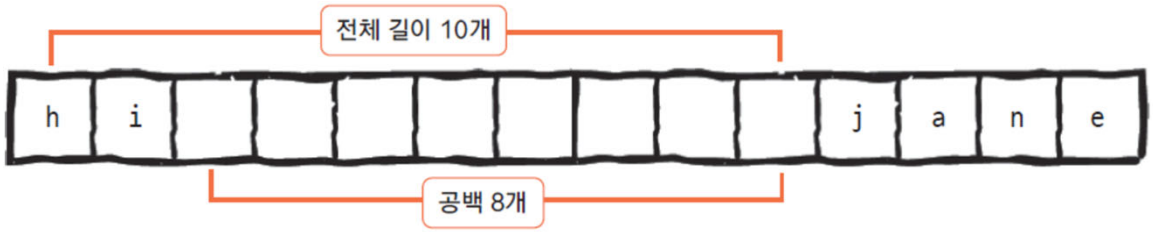
1. 정렬과 공백 표현하기

Do it! 코딩해 보세요

```
System.out.println(String.format("%-10sjane.", "hi")); // "hi       jane." 출력
```

실행 결과

```
hi       jane.
```



03-4 문자열

▪ 문자열 포맷 코드

▪ 문자열 포맷 코드 응용하기

2. 소수점 표현하기

Do it! 코딩해 보세요

```
System.out.println(String.format("%.4f", 3.42134234));
```

실행 결과

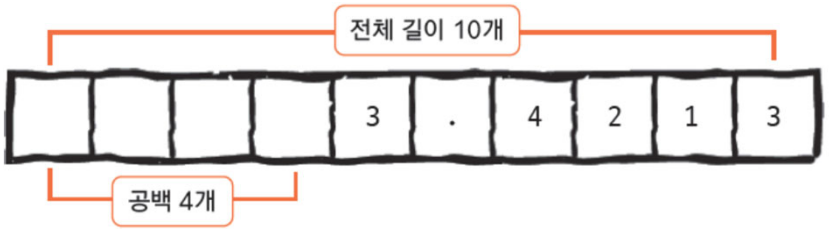
```
3.4213
```

Do it! 코딩해 보세요

```
System.out.println(String.format("%10.4f", 3.42134234));
```

실행 결과

```
    3.4213
```



03-4 문자열

▪ 문자열 포맷 코드

▪ System.out.printf

- System.out.printf 메서드를 사용하면 String.format 메서드가 없어도 같은 형식으로 간단히 포매팅된 문자열을 출력할 수 있음

Do it! 코딩해 보세요

```
System.out.printf("I eat %d apples.", 3);  // "I eat 3 apples." 출력
```

실행 결과

```
I eat 3 apples.
```



감사합니다.